Cookie API — Thought Process

Architecture

I split the application into four modules: Auth, Users, Cookies, CookieJar.

AuthModule handles signup, login, JWT creation, and authentication logic.

UsersModule handles user lookup and persistence

CookiesModule contains the protected user resource

CookieJarModule is for admin-only access

I kept UsersModule separate from AuthModule because Auth needs to look up users, but user management should remain its own responsibility

Authentication

Used JWT bearer tokens because the assignment asks for bearer token

Passwords are hashed before being saved

Guards and authorization

Protected routes use JwtAuthGuard, which validates the bearer token and attaches the user to the request.

For role-based access, I added a RolesGuard the flow is:

Request → JwtAuthGuard → RolesGuard → Controller

The cookieJar is restricted to admin only

Validation and error handling

DTOs use class-validator decorators to define validation rules. A global ValidationPipe enforces these rules on every request to strip unknown fields (whitelist) and convert types (transform).

Duplicate email registration is handled in auth.service

Security

Passwords are hashed before saving
Bearer token protect private resources
Role guards restrict admin-only routes

Tradeoffs

Refresh tokens I did not add refresh tokens because they were outside the assignment scope. In a production system, I would use both short lived access token and long lived refresh token

rate limiting. Right now nothing stops someone from hitting the login endpoint thousands of times

TypeORM schema synchronization automatically updates the database tables to match the entity files on every startup. For production, I would use database migrations instead

The admin user is created on startup. This works for the assignment, but a proper seed script or migration would be cleaner.

Testing

Added unit tests that mock dependencies to test business logic

End-to-end cover all HTTP requests

I also tested that password hashes are not returned in API responses just to be safe